

Algoritma dan Pemrograman Komputer 1

Pertemuan 4: Struktur Kontrol 2

M. Ma'ruf Qomaruddin Kafi – 5002241095
Thalita Ayu Antoniasari – 5002251011

15 Oktober, 2026

Departemen Matematika
Fakultas Sains dan Analitika Data
Institut Teknologi Sepuluh Nopember

Daftar Isi

Pengenalan Pernyataan SWITCH

Sintaks Dasar SWITCH

SWITCH dengan Berbagai Tipe Data

Pentingnya BREAK Statement

Multiple Case Statements

Nested SWITCH Statements

Best Practices SWITCH

Latihan

Kesimpulan

Referensi

Pengenalan Pernyataan SWITCH

Pernyataan SWITCH dalam Java

Definisi

Pernyataan `switch` adalah struktur kontrol percabangan yang digunakan ketika kita memiliki banyak kondisi yang identik dan ingin membandingkan suatu ekspresi dengan beberapa nilai konstan.

Keunggulan SWITCH

- Lebih rapi dan mudah dibaca dibanding multiple if-else
- Memudahkan pemilihan berdasarkan nilai yang spesifik
- Mendukung tipe tertentu, seperti `char`, `byte`, `short`, `int`, dan `String` (JDK 7+)

Perbandingan SWITCH dan IF-ELSE IF

Menggunakan IF-ELSE

```
1  char nilai = 'B';
2
3  if (nilai == 'A') {
4      System.out.println("Sangat Baik");
5  } else if (nilai == 'B') {
6      System.out.println("Baik");
7  } else if (nilai == 'C') {
8      System.out.println("Cukup");
9  } else {
10     System.out.println("Nilai lainnya");
11 }
12
```

Menggunakan SWITCH

```
1  char nilai = 'B';
2
3  switch (nilai) {
4      case 'A':
5          System.out.println("Sangat Baik");
6          break;
7      case 'B':
8          System.out.println("Baik");
9          break;
10     case 'C':
11         System.out.println("Cukup");
12         break;
13     default:
14         System.out.println("Nilai lainnya");
15 }
16
```

Kesimpulan

Keduanya menghasilkan output yang sama. `if-else` lebih fleksibel untuk kondisi kompleks, sedangkan `switch` cocok untuk membandingkan satu nilai dengan beberapa pilihan tetap.

Perbandingan SWITCH vs IF-ELSE IF

SWITCH Statement	IF-ELSE IF Statement
Cocok untuk pengecekan nilai tetap	Cocok untuk pengecekan range
Lebih rapi untuk banyak pilihan	Lebih fleksibel untuk kondisi rumit
Berfokus pada nilai yang cocok	Mudah ditambah kondisi baru
Perlu `break` untuk setiap case	Tidak perlu `break`
Umumnya untuk: char, int, String	Untuk semua tipe data dan ekspresi

Tabel 1: Perbandingan SWITCH vs IF-ELSE IF

Sintaks Dasar SWITCH

Sintaks Dasar Pernyataan SWITCH

Struktur Dasar SWITCH

```
switch (ekspresi) {
    case nilai1:
        pernyataan1;
        break;
    case nilai2:
        pernyataan2;
        break;
    case nilai3:
        pernyataan3;
        break;
    default:
        pernyataanDefault;
        break;
}

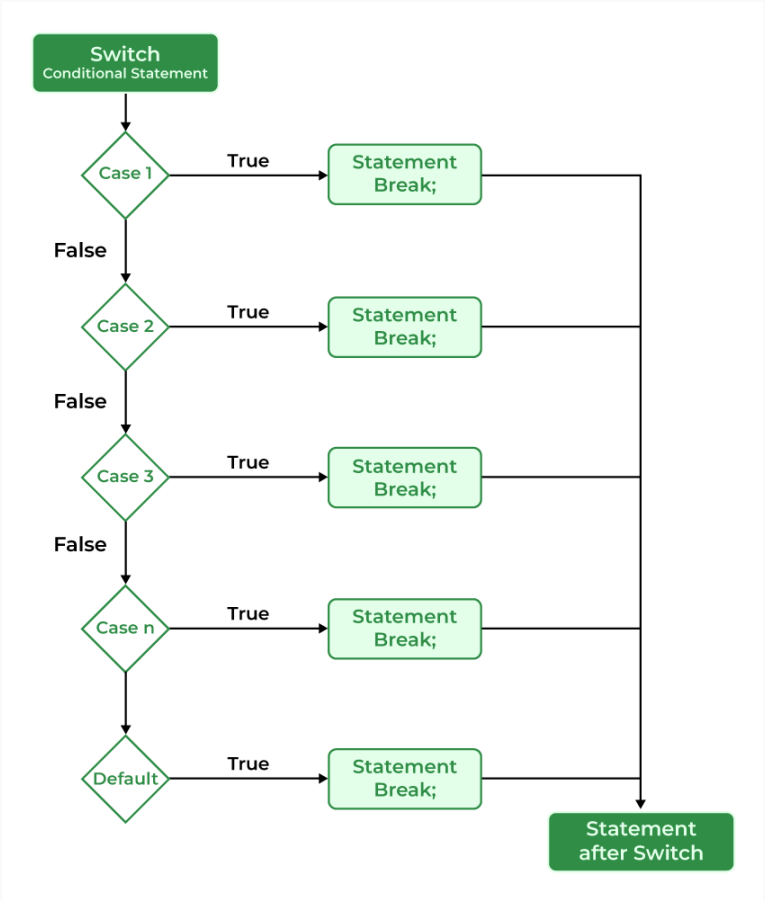
```

Komponen SWITCH Statement

Komponen	Deskripsi
`switch (ekspresi)`	Ekspresi yang dievaluasi; pada `switch` klasik, tipe yang umum digunakan antara lain `char`, `byte`, `short`, `int`, dan `String`
`case nilai:`	Nilai konstanta yang dibandingkan dengan ekspresi
`pernyataan;`	Blok kode yang dieksekusi jika case match
`break;`	Menghentikan eksekusi dan keluar dari switch
`default:`	Blok yang dieksekusi jika tidak ada case yang match

Tabel 2: Komponen-komponen dalam pernyataan SWITCH

Flowchart Pernyataan SWITCH



Gambar 1: Alur eksekusi pernyataan SWITCH dalam Java

Sumber: Programiz - Java switch Statement

SWITCH dengan Berbagai Tipe Data

SWITCH dengan Tipe Data INT

Contoh Program: Konversi Bulan

```
1  public class SwitchInt {
2  ...
3      public static void main(String[] args) {
4          int bulan = 8;
5          String bulanString;
6
7          switch (bulan) {
8              case 1: bulanString = "Januari"; break;
9              case 2: bulanString = "Februari"; break;
10             case 3: bulanString = "Maret"; break;
11             case 4: bulanString = "April"; break;
12             case 5: bulanString = "Mei"; break;
13             case 6: bulanString = "Juni"; break;
14             case 7: bulanString = "Juli"; break;
15             case 8: bulanString = "Agustus"; break;
16
```

SWITCH dengan Tipe Data INT

Contoh Program: Konversi Bulan

```
            case 9: bulanString = "September"; break;
            case 10: bulanString = "Oktober"; break;
            case 11: bulanString = "November"; break;
            case 12: bulanString = "Desember"; break;
            default: bulanString = "Bulan tidak valid"; break;
        }
        System.out.println(bulanString);
    }
}

```

Output SWITCH dengan INT

Contoh Eksekusi Program

```
Agustus
```

Penjelasan

- Variabel `bulan` bernilai 8
- Program mencari case yang match dengan nilai 8
- Ditemukan di `case 8:` dan mengeksekusi `bulanString = "Agustus"`
- `break` menghentikan eksekusi ke case berikutnya

SWITCH dengan Tipe Data CHAR

Contoh Program: SWITCH dengan Char

```
1  import java.util.Scanner;
2  // import java.io.IOException;  // Untuk alternatif System.in.read()
3
4  public class SwitchChar {
5      public static void main(String[] args) {
6          Scanner scanner = new Scanner(System.in);
7          System.out.println("Masukkan inisial nama anda:");
8
9          // Mengambil karakter pertama dari input
10         char inisial = scanner.next().charAt(0);
11         // Bisa juga menggunakan:
12         // char inisial = (char)System.in.read(); (dengan throws IOException)
13
```

SWITCH dengan Tipe Data CHAR

Contoh Program: SWITCH dengan Char

```
1         switch(Character.toLowerCase(inisial)) {
2             case 'a':
3                 System.out.println("Nama anda pasti Angga");
4                 break;
5             case 'b':
6                 System.out.println("Nama anda pasti Budi");
7                 break;
8             case 'c':
9                 System.out.println("Nama anda pasti Cici");
10                break;
11            default:
12                System.out.println("Nama anda tidak terkenal");
13                break;
14        }
15    }
16}
17
```

Penjelasan Input Char

Cara Kerja `scanner.next().charAt(0)`

- `scanner.next()` membaca input sebagai **String**
- `.charAt(0)` mengambil **karakter pertama** dari string tersebut
- Contoh: Input "Budi" $\rightarrow$ `charAt(0)` $=$ 'B'

Alternatif: `System.in.read()`

- Membaca langsung 1 karakter dari input stream
- Butuh `throws IOException` di method signature
- Lebih "pure" untuk input karakter tunggal

Output SWITCH dengan CHAR

Contoh Eksekusi Program

```
Masukkan karakter nama anda:  a
Nama anda pasti Angga
```

Contoh Lain

```
Masukkan karakter nama anda:  c
Nama anda pasti Cici
```

Catatan Penting

- Gunakan single quote (' ') untuk karakter
- `System.in.read()` membutuhkan `throws IOException`
- `Character.toLowerCase()` membuat program case-insensitive. 'A' dan 'a' akan diperlakukan sama

SWITCH dengan Tipe Data STRING

Contoh Program: SWITCH dengan String

```
1  public class SwitchString {
2      public static void main(String[] args) {
3          String hari = "Senin";
4          String jenisHari;
5
6          switch (hari.toLowerCase()) {
7              case "senin":
8              case "selasa":
9              case "rabu":
10             case "kamis":
11             case "jumat":
12                 jenisHari = "Hari kerja"; break;
13
```

SWITCH dengan Tipe Data STRING

Contoh Program: SWITCH dengan String

```
            case "sabtu":
            case "minggu":
                jenisHari = "Hari libur"; break;
            default:
                jenisHari = "Hari tidak valid"; break;
        }
        System.out.println(jenisHari);
    }
}

```

Output SWITCH dengan STRING

Contoh Eksekusi Program 1: Hari Kerja

```
String hari = "Senin";
Hari kerja
```

Contoh Eksekusi Program 2: Hari Libur

```
String hari = "Sabtu";
Hari libur
```

Penjelasan

- `hari.toLowerCase()` mengubah "Senin" menjadi "senin" agar pencocokan case bersifat *case-insensitive*
- **Multiple case** (`"senin"`, `"selasa"`, ...) dikelompokkan untuk aksi yang sama: `"Hari kerja"`
- `"sabtu"` dan `"minggu"` menghasilkan `"Hari libur"`
- Jika input di luar daftar (misal `"holiday"`), `default` akan mengeksekusi `"Hari tidak valid"`

Pentingnya BREAK Statement

Konsekuensi Tanpa BREAK

SWITCH Tanpa BREAK - Fall Through

```
1  import java.io.IOException;
2
3  public class SwitchNoBreak {
4      public static void main(String[] args) throws IOException {
5          System.out.println("Masukkan karakter nama anda");
6          char inisial = (char)System.in.read();
7
8          switch(inisial) {
9              case 'a': System.out.println("Nama anda pasti Angga");
10             case 'b': System.out.println("Nama anda pasti Budi");
11             default: System.out.println("Nama anda tidak terkenal");
12         }
13     }
14 }
15
```

Output SWITCH Tanpa BREAK

Contoh Eksekusi Program

```
Masukkan karakter nama anda:  a
Nama anda pasti Angga
Nama anda pasti Budi
Nama anda tidak terkenal
```

Fenomena Fall Through

- Tanpa `break`, eksekusi akan "jatuh" ke case berikutnya
- Semua pernyataan setelah case yang match akan dieksekusi
- Dapat dimanfaatkan untuk multiple case dengan aksi sama

Multiple Case Statements

Multiple Case untuk Aksi yang Sama

Contoh Program: Jumlah Hari dalam Bulan

```
1  public class SwitchMultipleCase {
2      public static void main(String[] args) {
3          int bulan = 2;
4          int tahun = 2000;
5          int jmlHari = 0;
6
7          switch (bulan) {
8              case 1: case 3: case 5: case 7:
9              case 8: case 10: case 12:
10                 jmlHari = 31;
11                 break;
12             case 4: case 6: case 9: case 11:
13                 jmlHari = 30;
14                 break;
15
```

Multiple Case untuk Aksi yang Sama

Contoh Program: Jumlah Hari dalam Bulan

```
            case 2:
                if (((tahun % 4 == 0) && !(tahun % 100 == 0))
                    || (tahun % 400 == 0))
                    jmlHari = 29;
                else
                    jmlHari = 28;
                break;
            default:
                System.out.println("Bulan tidak valid.");
                break;
        }
        System.out.println("Jumlah hari = " + jmlHari);
    }
}

```

Output Multiple Case

Contoh Eksekusi Program

```
Jumlah hari = 29
```

Penjelasan

- Bulan 2 (Februari) dengan tahun 2000 (tahun kabisat)
- Tahun kabisat: habis dibagi 4, tapi tidak habis dibagi 100, atau habis dibagi 400
- 2000 habis dibagi 400 $\rightarrow$ tahun kabisat $\rightarrow$ 29 hari

Nested SWITCH Statements

Konsep Nested SWITCH

SWITCH di dalam SWITCH

- `switch` dapat diletakkan di dalam blok `case` dari `switch` lain.
- Berguna untuk menu bertingkat atau klasifikasi kompleks.
- `break` hanya keluar dari `switch` tempat `break` tersebut berada.

Contoh Program Nested SWITCH

```java
import java.util.Scanner;

public class NestedSwitch {
public static void main(String[] args) {
    Scanner baca = new Scanner(System.in);

    System.out.println("Pilih kategori:");
    System.out.println("1. Minuman");
    System.out.println("2. Makanan");
    System.out.print("Pilihan: ");
    int kategori = baca.nextInt();

    switch (kategori) {
    case 1:
        System.out.println("1. Teh");
        System.out.println("2. Kopi");
        System.out.print("Pilih minuman: ");
        int minuman = baca.nextInt();

        switch (minuman) {
        case 1:
            System.out.println("Anda memilih teh");
            break;
        case 2:
            System.out.println("Anda memilih kopi");
            break;
        default:
            System.out.println("Pilihan minuman tidak valid");
        }
        break;

```

Contoh Program Nested SWITCH (Lanjutan)

```
    case 2:
        System.out.println("1. Nasi goreng");
        System.out.println("2. Mi goreng");
        System.out.print("Pilih makanan: ");
        int makanan = baca.nextInt();

        switch (makanan) {
        case 1:
            System.out.println("Anda memilih nasi goreng");
            break;
        case 2:
            System.out.println("Anda memilih mi goreng");
            break;
        default:
            System.out.println("Pilihan makanan tidak valid");
        }
        break;
    default:
        System.out.println("Kategori tidak valid");
        break;
    }
    baca.close();
}
}

```

Best Practices SWITCH

Best Practices menggunakan SWITCH

Tips dan Rekomendasi

- **Selalu gunakan break** kecuali untuk fall through yang disengaja
- **Gunakan default case** untuk menangani nilai tak terduga
- **Group case yang sama** untuk menghindari duplikasi kode
- **Pertimbangkan readability** - jika terlalu kompleks, gunakan if-else

Kapan Memilih SWITCH vs IF-ELSE?

- **SWITCH**: Untuk equality check dengan nilai konstan yang terbatas
- **IF-ELSE**: Untuk range check, kondisi kompleks, atau banyak kondisi

Latihan

Soal 1. Klasifikasi Nilai Akademik

Soal 1

Buat program untuk mengubah nilai angka menjadi huruf mutu beserta deskripsinya sesuai Peraturan Akademik ITS.

1. Minta input nilai angka (0–100).
2. Gunakan `if-else` untuk menentukan huruf mutu:
   - 86–100 $\rightarrow$ A
   - 76–85 $\rightarrow$ AB
   - 66–75 $\rightarrow$ B
   - 61–65 $\rightarrow$ BC
   - 56–60 $\rightarrow$ C
   - 41–55 $\rightarrow$ D
   - 0–40 $\rightarrow$ E
3. Gunakan `switch` pada huruf mutu untuk menampilkan deskripsi (A=Istimewa, AB=Baik Sekali, B=Baik, BC=Cukup Baik, C=Cukup, D=Kurang, E=Kurang Sekali).
4. Tampilkan **nilai angka**, **huruf mutu**, dan **deskripsi**.

Soal 1. Contoh Output

Contoh 1: Nilai Tengah

```
Masukkan nilai (0-100):  82
Nilai angka :  82
Huruf mutu :  AB
Deskripsi :  Baik Sekali
```

Contoh 2: Nilai Batas Bawah

```
Masukkan nilai (0-100):  40
Nilai angka :  40
Huruf mutu :  E
Deskripsi :  Kurang Sekali
```

Soal 2. Menu Kantin ITS (Nested SWITCH)

Soal 2

Buat program dengan **nested SWITCH** untuk simulasi pemesanan di kantin ITS.

- **Level 1 – Pilih kategori:**
  1. Makanan
  2. Minuman
  3. Snack
- **Level 2 – Pilih menu sesuai kategori:**
  - Makanan: Nasi Goreng (Rp15000), Mie Ayam (Rp12000), Ayam Geprek (Rp13000)
  - Minuman: Es Teh (Rp5000), Es Jeruk (Rp7000), Kopi (Rp6000)
  - Snack : Gorengan (Rp3000), Risol (Rp4000)
- Tampilkan nama pesanan dan harganya.
- Gunakan `default` untuk menangani pilihan tidak valid, baik di level 1 maupun level 2.

Soal 2. Contoh Output

Contoh 1: Pesanan Valid

```
=== KANTIN ITS ===
1. Makanan
2. Minuman
3. Snack
Pilih kategori: 1
--- MENU MAKANAN ---
1. Nasi Goreng - Rp15000
2. Mie Ayam - Rp12000
3. Ayam Geprek - Rp13000
Pilih menu: 2
Pesanan : Mie Ayam
Harga : Rp12000
```

Soal 2. Contoh Output (Pilihan Tidak Valid)

Contoh 2: Kategori Salah

```
Pilih kategori:  5
Kategori tidak valid
```

Contoh 3: Menu Salah

```
Pilih kategori:  2
--- MENU MINUMAN ---
1.  Es Teh - Rp5000
2.  Es Jeruk - Rp7000
3.  Kopi - Rp6000
Pilih menu:  9
Pilihan menu tidak valid
```

Kesimpulan

Kesimpulan

Inti Pertemuan 4: Pertemuan 4: Struktur Kontrol 2/SWITCH

- **SWITCH** alternatif yang elegan untuk multiple if-else
- Mendukung berbagai tipe data: `char, int, String, dan lain-lain`
- **BREAK** statement penting untuk mencegah fall through
- **DEFAULT** case menangani nilai tak terduga
- **Multiple case** untuk grouping aksi yang sama
- **Nested switch** untuk menu bertingkat yang kompleks

Referensi

Referensi

Referensi:

- **Modul Praktikum Algoritma dan Pemrograman - Modul 6**
  Departemen Matematika FSAD ITS
- **Oracle Java Tutorials - The switch Statement**
  `https://docs.oracle.com/javase/tutorial/java/nutsandbolts/switch.html`
- **Programiz - Java switch Statement**
  `https://www.programiz.com/java-programming/switch-statement`
- **GeeksforGeeks - Switch Statements in Java**
  `https://www.geeksforgeeks.org/switch-statement-in-java/`

Terima Kasih

Pertanyaan dan Diskusi